

Multi-Cloud Wildlife-Image Job-Processing Service: Design

Lucas Huang

1 Overview, Assumptions and Key Decisions

Partners upload a 1–5 GB image batch; each job runs *prep* (CPU) → *species ID* (GPU) → *report* (CPU). The design rests on three ideas: (i) one transactional store (PostgreSQL) holds job state, the queue and leases, so there is never a dual-write between a queue and a database; (ii) workers *pull* leases carrying a monotonically increasing *fencing epoch*, and outputs are written to per-attempt immutable prefixes and published by a conditional update, so a result is published exactly once even if a worker reappears; (iii) *compute is active-active across AWS and GCP*, state is *active-passive*, and every job is processed *data-locally* in its home cloud to avoid egress.

Assumption	Value and consequence
Workload	100 jobs/day; burst 100/h; mean batch 3 GB ($\approx 1,000$ JPEGs); mean job 15 min split 5/6/4 min (prep/ID/report); prep output ≈ 5 –10% of input (resized images).
GPU is the bottleneck	4 GPUs $\times 60/6 = 40$ jobs/h. A 100-job burst drains in 2.5 h (worst case, all 30-min jobs: 5 h) $\Rightarrow$ durable queue, ETA shown to users, acceptance never blocks on capacity. Daily GPU use is 10 of 96 GPU-h ($\approx 10\%$) $\Rightarrow$ GPU pools scale 0–2 per cloud.
CPU sizing	Burst prep 100 $\times$ 5 min ≈ 9 VMs; report is GPU-bound (40 $\times$ 4 min ≈ 3 VMs) $\Rightarrow \approx 12$ of 16 CPU VMs at peak.
VM cap	Worker VMs only: per cloud 0–2 GPU + 1–8 CPU = 10 per cloud, 20 total . The control plane (API, DB, storage, K8s masters) is managed/serverless. If it must count, 1 VM per cloud is carved out, leaving 18 workers, which still covers peak demand.
Network	Partner $\rightarrow$ storage burst 300 GB/h ≈ 670 Mbps aggregate (direct multipart upload, not via the API). VM $\leftrightarrow$ object store ≥ 5 Gbps. AWS $\leftrightarrow$ GCP HA VPN (4 tunnels $\times 1.25$ Gbps) ≈ 2.5 Gbps usable, same-metro RTT < 5 ms.
Pricing (indicative)	Egress $\approx \$0.12$ /GB; GPU VM $\approx \$1.2$ /h; standard object storage $\approx \$0.025$ /GB-month. Always-on 4 GPUs $\approx \$3.5$ k/month vs. scale-to-demand (≈ 10 GPU-h/day + 1 warm GPU in business hours) $\approx \$0.8$ k/month, at the cost of a ≈ 5 min cold start.
Regions	Both clouds in Sydney: widest Australian GPU supply; data stays in Australia (camera traps can capture people, so images may be personal information).
Third parties	Partner OIDC IdP; provider-neutral DNS with health checks; a small serverless witness for DB leadership; GitHub Actions. Supplied programs are CLI tools (input dir $\rightarrow$ output dir, exit code); model weights (≈ 1 GB) are kept in our own buckets, so nothing is pulled from public hubs at run time.

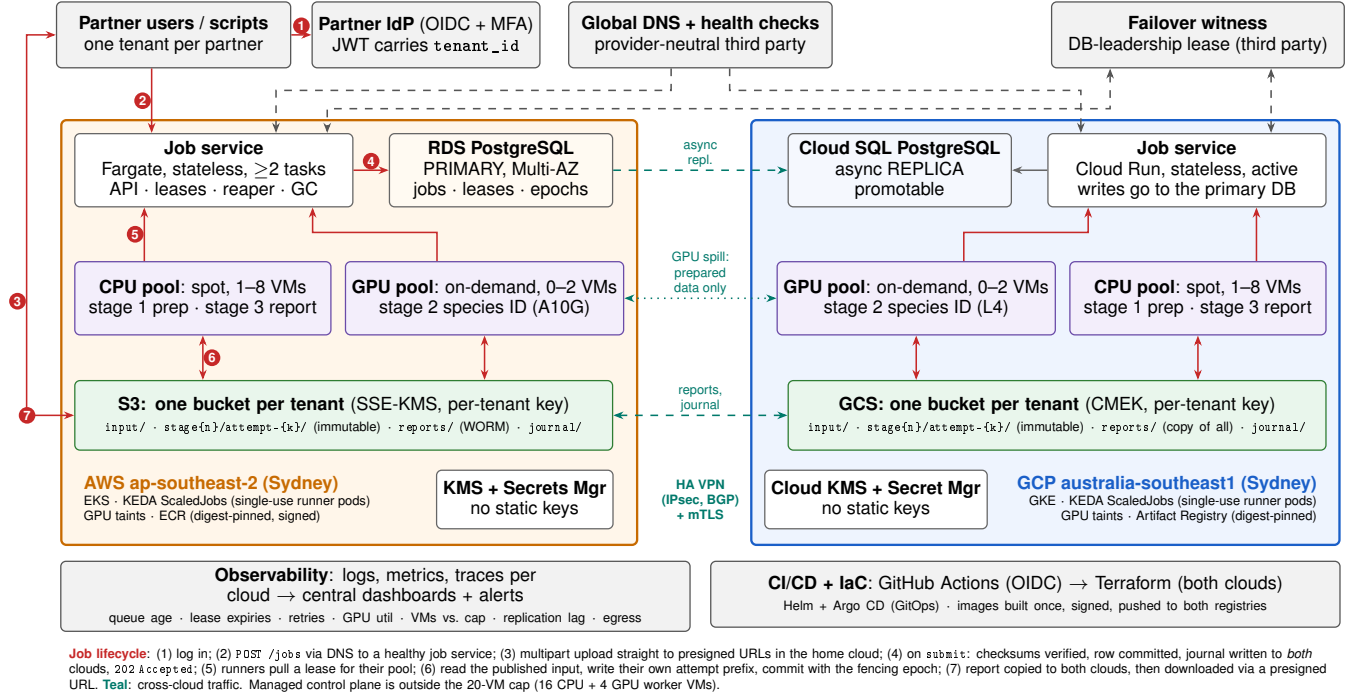


Figure 1: Architecture. Red numbers follow one job; teal links are the only cross-cloud traffic.

2 One Job's Lifecycle

States: UPLOADING $\rightarrow$ ACCEPTED $\rightarrow$ for each stage QUEUED $\rightarrow$ RUNNING $\rightarrow$ SUCCEEDED $\rightarrow$ COMPLETED; side exits FAILED_NEEDS_REVIEW, CANCELLED. (1) The partner logs in through its IdP and gets a JWT carrying *tenant_id*.

(2) `POST /jobs` (file list, sizes, SHA-256) reaches a healthy job service. The job service picks the *home cloud* (healthy, lowest estimated wait) and returns presigned multipart-upload URLs scoped to `jobs/{id}/input/` in the tenant bucket (15 min). (3) The client uploads directly; failed parts are retried individually. (4) `POST /jobs/{id}:submit` (with an `Idempotency-Key`) verifies checksums, commits `ACCEPTED + PREP_QUEUED`, writes a small acceptance record to *both* clouds' journals, and only then returns 202 with the job ID. (5) A CPU runner in the home cloud claims the prep lease; (6) it reads the input, writes `stage1/attempt-k/` and commits. That commit queues the GPU stage in the same transaction, and stage 3 follows the same way. (7) The report is copied to both clouds; only then is the job `COMPLETED`. A notification (transactional outbox) is sent, and `GET /jobs/{id}/report` returns a 5-minute presigned URL. `GET /jobs/{id}` shows the stage, queue position and ETA at any time.

3 Job Coordination and Placement

Acceptance and tracking. The `jobs` and `stage_runs` tables are the single source of truth. Each stage row holds `status`, `epoch`, `lease_owner`, `lease_expires_at` (database clock only, never worker clocks) and `published_attempt`. Load is <1 write/s, far below PostgreSQL limits, so a separate broker (SQS/PubSub) would add a dual-write hazard without benefit.

Worker discovery and assignment (pull). A small *runner agent* is baked into each stage image. It calls `POST /leases` with its capabilities (`cloud`, `stage`, `gpu`, `mem_gb`). The coordinator selects a row with `SELECT ...FOR UPDATE SKIP LOCKED` and, in the same transaction, sets `RUNNING`, increments `epoch`, and sets the lease to 90 s. It returns (`job`, `stage`, `attempt`, `epoch`) plus credentials scoped to that job (§9). Workers therefore need no registry: a worker exists while it is renewing leases, and VMs that never ask receive no work. Pull also balances load naturally, because idle runners take the next item.

Stage dependencies. A stage becomes `QUEUED` only inside the transaction that publishes its predecessor; downstream stages read only the *published* attempt pointer, never a prefix listing.

Resources and placement. At submission each job gets a size class (S/M/L by batch size), which maps to a memory requirement. A lease matches only if the runner's pool satisfies it: GPU stages run only on tainted GPU pools, and L-class prep runs on the 32 GB node group. Ordering is *weighted fair share across tenants*, then oldest first, with a per-tenant cap of 2 concurrent GPU stages, so one partner's 50-job burst cannot starve the others. **Data-local placement:** prep runs in the home cloud. The GPU stage *spills* to the other cloud only if $ETA_{home} > ETA_{other} + t_{transfer} + 20 \text{ min}$ (hysteresis); it then moves the *prepared* output ($\approx 150 \text{ MB}$, $\approx \$0.02$) rather than the raw 3 GB batch ($\approx \$0.36$). The report stage has small inputs and runs on whichever cloud has a free CPU runner.

4 VMs, Containers and Orchestration

Provisioning (IaC). Terraform modules per cloud (VPC, HA VPN, EKS/GKE, node pools, buckets with lifecycle rules, KMS, databases, IAM) plus a global stack (DNS, witness). State lives in an encrypted remote backend with locking. GitHub Actions runs `plan` on pull requests and `apply` after review, authenticating with OIDC federation (no cloud keys in CI). In-cluster objects are Helm charts synced by Argo CD (GitOps), so both clouds run identical, versioned manifests. Node pools enforce the cap in IaC (`max_size`: GPU 2, CPU 8 per cloud), and an alert fires if the VM count reaches 19. Spot is used for CPU (a preemption is just another lease expiry); GPU is on-demand (scarce, only 4 slots).

Containers. There is one image per stage: the supplied program + the runner agent, built once in CI, scanned, signed (cosign), pushed to both registries and deployed *by digest*. The GPU image pins the CUDA base, framework versions and driver range. Model weights are *not* baked in (the same pattern as my A1 SmartPark deployment): an init container fetches `model@version` from our bucket, verifies its SHA-256, and stores it in a node-local read-only cache, so restarts do not re-download it.

Isolation and scaling. KEDA *ScaledJobs* create one single-use pod per queued lease (trigger: queued count for that stage and cloud, from the coordinator's metrics endpoint). Each pod runs exactly one attempt and exits, so every tenant job gets a fresh filesystem. Pod requests ($\approx 90\%$ of a node) place one stage per VM. The cluster autoscaler adds and removes nodes within the pool bounds; GPU pools scale to zero after 10 min idle, except one warm GPU in business hours. Pods run as non-root with a read-only root filesystem, all capabilities dropped and `seccomp RuntimeDefault`. NetworkPolicy is default-deny; the only egress allowed is to the coordinator and private storage endpoints, so a third-party program cannot exfiltrate data.

5 Storage and ML Reproducibility

Data	Store	Durability, lifecycle and cleanup
Inputs	S3/GCS, bucket per tenant, home cloud	Object stores are designed for 11 nines; versioned. Deleted 30 days after the job ends (partners keep originals). Not replicated cross-cloud (9 TB/month would cost $\approx$ \$1,080/month in egress).
Intermediates	same bucket, <code>stage{n}/attempt-{k}/</code>	Immutable, <code>_MANIFEST.json</code> (files + SHA-256) written <i>last</i> . Unpublished attempts are GC'd after 24h; published ones 7 days after the job ends.
Reports	both clouds, <code>reports/</code>	Versioning + object lock (WORM, 1 year); copied to the second cloud <i>before COMPLETED</i> (RPO=0); Infrequent Access tier after 30 days.
Metadata	RDS PostgreSQL (Multi-AZ) $\rightarrow$ Cloud SQL replica	PITR 14 days, daily snapshot, async logical replica in GCP (lag typically <5s). Acceptance journal in both clouds closes the lag gap.

Provenance. Each published stage run stores an immutable record: input-manifest hash, image digest, program version, model name/version/SHA-256, framework/CUDA/driver versions, instance type, cloud/region, attempt, epoch, timings, exit code and output-manifest hash. The report embeds a provenance digest, so a result can be re-run with exactly the same artefacts. Images and model versions referenced by retained reports are exempt from registry clean-up.

6 Health and Recovery

Readiness. The GPU pod's startup probe passes only after the model is loaded, a CUDA self-test succeeds and storage is reachable. A runner claims leases only when ready. The node health is taken from the kubelet and the GPU device plugin (Xid errors). **Crashes.** A non-zero exit or OOM is reported as a *program failure*. A lost pod, VM, preemption or partition shows up as missing heartbeats (every 30s); the reaper expires the lease after 90s, increments the epoch and requeues the stage on another node. **Stalls.** Heartbeats carry progress (child CPU time, bytes written). If progress is unchanged for 10 min, or a per-stage deadline of $2\times$ the historical p99 for the size class is exceeded, the attempt is killed and requeued. The agent also *self-fences*: if it cannot renew its lease, it stops the program before the lease expires and writes nothing more.

Repeated failures. Infrastructure failures do not consume the job's retry budget; they lower the *node's* health score, and 2 failures from different jobs within 1 h cordon and replace that node. Program failures are retried at most twice with exponential backoff; after an OOM, the retry goes to the next memory class. After that the job becomes `FAILED_NEEDS_REVIEW` (poison job): the input is kept, the partner and operator are notified, and the queue continues.

Logs and metrics. Structured JSON logs carry `job_id`, `tenant_id`, `stage`, `attempt`, `epoch`, `node`, `image digest`, `model hash`, and an OpenTelemetry trace spans the three stages. Alerts: oldest queued age >3h, lease expiries/h, retry rate, stage p50/p99 by size class, GPU utilisation and memory, VM count vs. cap, DB replication lag >60s, egress bytes/day, per-tenant failure rate.

7 Failure Scenario: GPU VM Lost After Writing Its Output

Time	Event and state change
t_0	G1 holds the stage-2 lease of job J with <code>epoch=7</code> ; it writes <code>stage2/attempt-7/</code> and then <code>_MANIFEST.json</code> .
t_1	G1 becomes unreachable before <code>commit</code> ; its heartbeats stop. Its scoped credentials expire with the lease.
$t_1+90\text{ s}$	The reaper, in one transaction, sets <code>epoch=8</code> , attempt 7 <code>ABANDONED</code> , stage <code>QUEUED</code> . <i>Optimisation</i> : if attempt 7's manifest exists and every checksum verifies, the reaper <i>adopts</i> it and publishes attempt 7 under <code>epoch=8</code> (saves ≈ 6 GPU-min); otherwise G2 claims the lease.
t_2	G2 writes <code>stage2/attempt-8/</code> and commits: <code>UPDATE stage_runs SET status='SUCCEEDED', published_attempt=8 WHERE job=J AND stage=2 AND epoch=8 AND status='RUNNING' $\rightarrow$ 1 row</code> ; stage 3 is queued in the same transaction.
t_3	G1 returns and commits with <code>epoch=7 $\rightarrow$ 0 rows $\rightarrow$ 409 STALE_EPOCH</code> . The agent discards its work and exits. Kubernetes has already deleted the pod, so the kubelet kills it on reconnect.
$t_3+24\text{ h}$	GC deletes the unpublished <code>attempt-7/</code> prefix.

Why there is no duplicate or conflicting result: (a) attempts never share a path, so G1 cannot overwrite G2; (b) exactly one attempt per stage can satisfy the epoch predicate; (c) consumers follow the published pointer only, so an orphan is invisible; (d) the report pointer and the outbox notification are written in the same transaction as `COMPLETED`, so a stale worker can never trigger a second report or e-mail; (e) the lease clock is the database clock, so G1's clock drift cannot help it.

8 Multi-Cloud Operation

Strategy: active-active compute, active-passive state (warm standby). Both clouds run job services and workers every day, so failover capacity is always warm and tested. The only singleton is the database writer (AWS);

GCP holds an async replica that can be promoted. A fully active-active multi-master DB (e.g. CockroachDB) would need ≥ 3 regions for quorum and a third-party dependency, which is too much for 100 jobs/day.

Connectivity and data movement. An AWS Site-to-Site VPN $\leftrightarrow$ GCP HA VPN link (4 IPsec tunnels, BGP, non-overlapping CIDRs) carries control traffic with mTLS on top. Over it go: DB logical replication; GCP job-service writes to the primary DB; report and journal copies; and rare GPU spills. Bulk inputs never cross. Cross-cloud reads use short-lived presigned URLs, so no long-lived cross-cloud IAM trust is needed. **Egress:** data-local placement cuts cross-cloud bytes from ≈ 9 TB/month ($\approx \$1,080$) to ≈ 30 GB/month ($\approx \4). A dedicated interconnect is not justified at this volume.

Dynamic failover (AWS unreachable). (1) *Detection:* external DNS health checks from several locations, plus GCP probes, fail for 3 min. (2) *No split-brain:* the AWS coordinator must renew a 60 s leadership lease in the witness. If it cannot, it stops granting leases and accepting writes. GCP waits $2\times$ that lease before promoting Cloud SQL and increments a *control-plane generation*; every request carries the generation, so a reconnecting old primary is refused. (3) DNS sends all API traffic to GCP; the GCP coordinator replays both journals, re-creating any accepted job lost to replication lag ($RPO=0$ for accepted jobs), and extends all running GCP leases by one grace period. Workers retry buffered commits with idempotency keys. *Keeps operating:* new submissions (homed in GCP), all GCP-homed jobs, status for every job, and download of every report. *Paused, not lost:* AWS-homed jobs whose inputs are in S3; they resume when AWS returns. Capacity drops to 10 VMs (20 jobs/h on GPU), because the unreachable cloud's VMs might still be running and must still count against the cap. $RTO \approx 5$ min. **Failback:** the old AWS database is rebuilt as a replica of GCP, then switched back in a quiet window. A GCP outage is simpler: AWS stays primary and only GCP-homed jobs pause.

9 Cloud Security

Users. Partners sign in through OIDC with MFA (their own IdP federated, or our user pool) and receive 1 h JWTs carrying `tenant_id` and a role (submitter/viewer/admin). Automation uses OAuth client credentials per partner. **Services.** Workloads use workload identity (EKS Pod Identity, GKE Workload Identity), so no static cloud keys exist anywhere. Service-to-service calls use mTLS with SPIFFE-style identities, and the few cross-cloud API calls use workload-identity federation.

Access control (defence in depth). (1) The API derives the tenant from the token, never from the request body. (2) PostgreSQL row-level security on `tenant_id` (`SET app.tenant_id` per transaction). (3) One bucket and one KMS key per tenant per cloud. (4) Per-lease credentials minted via STS session policies / GCP credential access boundaries: read `input/` and the published predecessor, write only this attempt's prefix, $TTL = \text{lease}$. (5) Presigned URLs are per object and short-lived. Deleting a tenant's key crypto-shreds its data when the partnership ends.

Secrets. Secrets live in AWS Secrets Manager / GCP Secret Manager, read at run time through workload identity and rotated every 30 days. DB access uses IAM authentication. Secret scanning runs in CI; images contain no secrets.

Network isolation. Workers sit in private subnets without public IPs and reach storage through private endpoints (S3 gateway endpoint, Private Google Access). Databases have no public endpoint. The API sits behind a WAF and rate limits, and admission control (Kyverno / Binary Authorization) admits only signed images. **Encryption.** TLS 1.2+ externally, mTLS internally, and IPsec between clouds (two layers). At rest: SSE-KMS/CMEK with per-tenant keys for objects, KMS-encrypted DB, disks and backups. Audit trail: CloudTrail data events, GCS data-access logs, and DB audit of all state transitions.

10 Trade-offs and Limitations

Async DB replication trades a few seconds of metadata RPO for write latency (the journal compensates). Not replicating inputs saves $\approx \$1k$ /month but pauses home-cloud jobs during an outage. Scale-to-zero GPUs save $\approx \$2.7k$ /month for a ≈ 5 min cold start. The witness and DNS are third-party dependencies: if both fail, failover becomes a manual, operator-approved promotion.